

Лабораторная работа по MySQL №2

Логические операторы

Мы рассмотрим, как выбрать данные на основе условий SQL, представленных с помощью булевых (логических) операторов.

- AND
- OR
- NOT

Использовать их очень просто. Ниже показан оператор **SELECT**, который выводит имена сотрудников, которые получают более 70000, но меньше 90000.

```
SELECT f_name, l_name from employee_data where salary > 70000 AND salary < 90000;
```

Давайте выведем список сотрудников, фамилии которых начинаются с буквы К или Л.

```
SELECT l_name from employee_data where l_name like 'К%' OR l_name like 'Л%';
```

Вот более сложный пример ... список имен и возраста сотрудников, фамилии которых начинаются с К или Л, и которые младше 30 лет.

```
SELECT f_name, l_name , age from employee_data where  
(l_name like 'К%' OR l_name like 'Л%') AND age < 30;
```

Обратите внимание на использование скобок в представленном выше операторе. Скобки предназначены для выделения различных логических условий и удаления двусмысленностей.

Оператор **NOT** поможет при поиске всех сотрудников, которые не являются программистами. (Программисты включают старших программистов, программистов мультимедиа и программистов).

```
SELECT f_name, l_name, title from employee_data where title NOT LIKE "%программист%";
```

Показать всех сотрудников со стажем работы в компании более 3 лет, которые старше 30 лет.

```
select f_name, l_name from employee_data where yos > 3 AND age > 30;
```

Операторы IN и BETWEEN

Чтобы найти сотрудников, которые являются разработчиками Web или системными администраторами, можно использовать оператор **SELECT** следующего вида:

```
SELECT f_name, l_name, title from employee_data where  
title = 'разработчик Web' OR title = 'системный адм.';
```

В SQL имеется более простой способ сделать это с помощью оператора **IN** (в множестве). Его использование не представляет никаких трудностей.

```
SELECT f_name, l_name, title from employee_data where  
title IN ('разработчик Web', 'системный адм.');
```

Использование **NOT** перед **IN** позволяет вывести данные, которые не входят в множество, определяемое условием **IN**. Следующий оператор выводит список сотрудников, которые не занимают должность программиста или системного администратора.

```
SELECT f_name, l_name, title from employee_data where  
title NOT IN ('программист', 'системный адм.');
```

Оператор **BETWEEN** используется для определения целочисленных границ. Поэтому вместо `age >= 32 AND age <= 40` можно использовать `age BETWEEN 32 AND 40`.

```
select f_name, l_name, age from employee_data where age BETWEEN 32 AND 40;
```

NOT также можно использовать вместе с **BETWEEN**, как в следующем операторе, который выводит сотрудников, зарплата которых меньше 90000 или больше 150000.

```
select f_name, l_name, salary from employee_data where  
salary NOT BETWEEN 90000 AND 150000;
```

Упорядочивание данных

Рассмотрим вопрос о том, как можно изменить порядок вывода данных, извлеченных из таблиц MySQL, используя предложение `ORDER BY` оператора `SELECT`.

Извлекаемые до сих пор данные всегда выводились в том порядке, в котором они были сохранены в таблице. В действительности SQL позволяет сортировать извлеченные данные с помощью предложения `ORDER BY`. Это предложение требует имя столбца, на основе которого будут сортироваться данные. Давайте посмотрим, как можно вывести имена сотрудников с упорядоченными по алфавиту фамилиями сотрудников (в возрастающем порядке).

```
SELECT l_name, f_name from employee_data ORDER BY l_name;
```

А вот так сотрудников можно отсортировать по возрасту.

```
SELECT f_name, l_name, age from employee_data ORDER BY age;
```

Предложение `ORDER BY` может сортировать в возрастающем порядке (`ASCENDING` или `ASC`) или в убывающем порядке (`DESCENDING` или `DESC`) в зависимости от указанного аргумента.

Чтобы вывести список сотрудников в убывающем порядке, можно использовать следующий оператор.

```
SELECT f_name from employee_data ORDER by f_name DESC;
```

Примечание: Возрастающий порядок (`ASC`) используется по умолчанию.

Ограничение количества извлекаемых данных

Далее рассмотрим, как ограничить число записей, выводимых оператором `SELECT`.

По мере увеличения таблиц возникает необходимость вывода только подмножества данных. Этого можно добиться с помощью предложения `LIMIT`.

Например, чтобы вывести из таблицы имена только первых пяти сотрудников, используется оператор `LIMIT` с аргументом равным 5.

```
SELECT f_name, l_name from employee_data LIMIT 5;
```

Это первые пять записей таблицы.

Можно соединить оператор `LIMIT` с оператором `ORDER BY`. Таким образом, следующий оператор выведет четверых самых молодых сотрудников компании.

```
SELECT f_name, l_name, age from employee_data ORDER BY age LIMIT 4;
```

Аналогично можно вывести двух самых старших сотрудников.

```
SELECT f_name, l_name, age from employee_data ORDER BY age LIMIT 2;
```

Извлечение подмножеств

`LIMIT` можно использовать также для извлечения подмножества данных, используя дополнительные аргументы.

Общая форма оператора `LIMIT` имеет следующий вид:

`SELECT (что-нибудь) from таблица LIMIT начальная строка, извлекаемое число записей;`

```
SELECT f_name, l_name from employee_data LIMIT 6,3;
```

Будут извлечены три строки, начиная с шестой.

Ключевое слово DISTINCT

Рассмотрим теперь, как выбрать и вывести записи таблиц MySQL с помощью ключевого слова DISTINCT (РАЗЛИЧНЫЙ), использование которого исключает появление повторяющихся данных.

Чтобы вывести все должности базы данных компании, можно выполнить следующий оператор:

```
select title from employee_data;
```

Можно видеть, что список содержит повторяющиеся данные. Предложение SQL DISTINCT выводит только уникальные данные. Вот как оно используется.

```
select DISTINCT title from employee_data;
```

Из этого можно видеть, что в компании имеется 11 уникальных должностей.

Уникальные записи можно также отсортировать с помощью ORDER BY.

```
select DISTINCT age from employee_data ORDER BY age;
```

DISTINCT часто используется вместе с функцией COUNT, которая будет рассмотрена далее.

Изменение записей

Команда UPDATE выполняет изменение данных в таблицах. Она имеет очень простой формат.

```
UPDATE имя_таблицы SET  
имя_столбца_1 = значение_1,  
имя_столбца_2 = значение_2,  
имя_столбца_3 = значение_3, ...  
[WHERE условия];
```

Как и все другие команды SQL можно вводить ее на одной строке или на нескольких строках.

Предположим, директор увеличил свою зарплату на 20000 и надбавки на 5000. Его предыдущая зарплата была 200000, а надбавки были 50000.

```
UPDATE employee_data SET salary=220000, perks=55000 WHERE title='директор';
```

Можно проверить эту операцию, выводя данные из таблицы.

```
select salary, perks from employee_data WHERE title = 'директор';
```

В действительности предыдущую зарплату знать не требуется. Можно воспользоваться арифметическими операторами. Следующий оператор сделает то же самое, при этом исходные данные знать заранее не требуется.

```
UPDATE employee_data SET salary = salary + 20000, perks = perks + 5000 WHERE title='директор';
```

В качестве другого примера можно попробовать изменить название должности "разработчик Web" на "программист Web".

```
update employee_data SET title = 'программист Web' WHERE title = 'разработчик Web';
```

Важно также перед выполнением изменений внимательно изучить часть оператора с условием, так как легко можно изменить не те данные. Оператор UPDATE без условий изменит все данные столбца во всех строках. Надо быть очень осторожным при внесении изменений.

Задания

1. Вывести имена и фамилии всех сотрудников, которые получают зарплату не более 90000 и не являются программистами, старшими программистами или программистами мультимедиа.
2. Что делает следующий оператор?

```
SELECT l_name, f_name from employee_data where title NOT LIKE '%продавец%' AND age < 30;
```

3. Вывести все идентификационные номера и имена сотрудников в возрасте между 32 и 40 годами.
4. Выберите имена всех сотрудников в возрасте 32 лет, которые не являются программистами.
5. Найдите всех сотрудников, которые занимают должность "старший программист" и "программист мультимедиа".
6. Выведите список имен сотрудников, зарплата которых составляет от 70000 до 90000.
7. Что делает следующий оператор?

```
SELECT f_name, l_name, title from employee_data where title NOT IN  
( 'программист', 'старший программист', 'программист мультимедиа' );
```

8. Вот более сложный оператор, который объединяет BETWEEN и IN. Что он делает?

```
SELECT f_name, l_name, title, age from employee_data where title NOT IN  
( 'программист', 'старший программист', 'программист мультимедиа' )  
AND age NOT BETWEEN 28 and 32;
```

9. Вывести список сотрудников в порядке, определяемом зарплатой, которую они получают.
10. Выведите список сотрудников в убывающем порядке их стажа работы в компании.
11. Что делает следующий оператор?

```
SELECT emp_id, l_name, title, age from employee_data ORDER BY title DESC, age ASC;
```

12. Вывести список сотрудников (фамилию и имя), которые занимают должность "программист" или "разработчик Web" и отсортировать их фамилии по алфавиту.
13. Найдите имена 5 самых молодых сотрудников компании.
14. Извлеките 5 записей, начиная с 10 строки.
15. Выведите имя и зарплату сотрудника, который получает самую большую зарплату.
16. Что делает следующий оператор?

```
SELECT emp_id, age, perks from employee_data ORDER BY perks DESC LIMIT 10;
```

17. Сколько уникальных вариантов зарплаты имеется в компании? Представьте их в убывающем порядке.
18. Сколько различных имен имеется в базе данных?
19. Измените фамилию Чащина на Петрова. Внесите соответствующие изменения в базу данных.
20. Название должности "программист мультимедиа" необходимо изменить на "специалист по мультимедиа".
21. Увеличьте зарплату всем сотрудниками (кроме директора) на 10000.